

Constraints in Oracle

What are Constraints?

Constraints in Oracle are rules applied to table columns to ensure **data accuracy, consistency, and integrity** in a database.

They restrict the type of data that can be inserted, updated, or deleted in a table.

Why Constraints are Important

- Prevent invalid data entry
- Avoid duplicate records
- Maintain relationships between tables
- Enforce business rules
- Improve overall data reliability

Types of Constraints in Oracle

1. NOT NULL Constraint

Description

The NOT NULL constraint ensures that a column **cannot contain NULL values**.

A value must be provided while inserting a record.

Syntax

column_name datatype NOT NULL

Example

```
CREATE TABLE EMPLOYEE (  
    EMP_ID NUMBER,  
    EMP_NAME VARCHAR2(50) NOT NULL  
);
```

In this example, EMP_NAME cannot be left empty.

2. UNIQUE Constraint

Description

The UNIQUE constraint ensures that **all values in a column are different**.

Oracle allows only **one NULL value** in a UNIQUE column.

Syntax

column_name datatype UNIQUE

Example

```
CREATE TABLE EMPLOYEE (  
    EMP_ID NUMBER,
```

EMAIL VARCHAR2(50) UNIQUE
);
No two employees can have the same email address.

3. PRIMARY KEY Constraint

Description

The PRIMARY KEY constraint:

- Uniquely identifies each record in a table
- Is a combination of NOT NULL and UNIQUE
- Allows only one primary key per table

Syntax (Column Level)

column_name datatype PRIMARY KEY

Syntax (Table Level)

CONSTRAINT constraint_name PRIMARY KEY (column_name)

Example

```
CREATE TABLE EMPLOYEE (  
    EMP_ID NUMBER PRIMARY KEY,  
    EMP_NAME VARCHAR2(50)  
);  
EMP_ID uniquely identifies each employee record.
```

4. FOREIGN KEY Constraint

Description

The FOREIGN KEY constraint maintains **referential integrity** between two tables.

It ensures that a value in a child table exists in the parent table.

Syntax

FOREIGN KEY (column_name)
REFERENCES parent_table(column_name)

Example

```
CREATE TABLE DEPARTMENT (  
    DEPT_ID NUMBER PRIMARY KEY,  
    DEPT_NAME VARCHAR2(30)  
);
```

```
CREATE TABLE EMPLOYEE (  
    EMP_ID NUMBER PRIMARY KEY,  
    EMP_NAME VARCHAR2(50),  
    DEPT_ID NUMBER,  
    CONSTRAINT fk_dept
```

```
FOREIGN KEY (DEPT_ID) REFERENCES DEPARTMENT(DEPT_ID)
);
```

An employee cannot be assigned to a department that does not exist.

5. CHECK Constraint

Description

The CHECK constraint ensures that column values satisfy a specified condition.

Syntax

CHECK (condition)

Example

```
CREATE TABLE EMPLOYEE (
    EMP_ID NUMBER,
    SALARY NUMBER(8,2),
    CONSTRAINT chk_salary CHECK (SALARY > 0)
);
```

Salary must always be greater than zero.

6. DEFAULT Constraint

Description

The DEFAULT constraint assigns a predefined value to a column when no value is supplied during insertion.

Syntax

column_name datatype DEFAULT value

Example

```
CREATE TABLE EMPLOYEE (
    EMP_ID NUMBER,
    STATUS VARCHAR2(10) DEFAULT 'ACTIVE'
);
```

If no value is provided for STATUS, Oracle automatically stores 'ACTIVE'.

Complete Example Using Multiple Constraints

```
CREATE TABLE EMPLOYEE (
    EMP_ID NUMBER,
    EMP_NAME VARCHAR2(50) NOT NULL,
    EMAIL VARCHAR2(50) UNIQUE,
    SALARY NUMBER(8,2),
    STATUS VARCHAR2(10) DEFAULT 'ACTIVE',
    DEPT_ID NUMBER,
    CONSTRAINT pk_emp PRIMARY KEY (EMP_ID),
    CONSTRAINT chk_salary CHECK (SALARY > 0));
```

Lab Exercise 1: Applying Column-Level Constraints

Aim:

To create a table using **column-level constraints** in Oracle to ensure data integrity.

Question:

Write SQL commands in Oracle to perform the following operations:

1. Create a table **EMPLOYEE** with the following attributes and constraints:
 - EMP_ID – NUMBER (Primary Key)
 - EMP_NAME – VARCHAR2(50) (NOT NULL)
 - EMAIL – VARCHAR2(50) (UNIQUE)
 - SALARY – NUMBER(8,2) (CHECK: Salary must be greater than 0)
 - STATUS – VARCHAR2(10) (DEFAULT value should be 'ACTIVE')
2. Insert at least **three valid records** into the EMPLOYEE table.
3. Attempt to insert a record with **NULL EMP_NAME** and observe the error.
4. Attempt to insert a record with **negative salary** and observe the error.

Lab Exercise 2: Applying Table-Level Constraints

Aim:

To implement **table-level constraints** using PRIMARY KEY, UNIQUE, and CHECK constraints.

Question:

Write SQL commands in Oracle to perform the following operations:

1. Create a table **PRODUCT** with the following attributes:
 - PRODUCT_ID – NUMBER
 - PRODUCT_NAME – VARCHAR2(50)
 - CATEGORY – VARCHAR2(30)
 - PRICE – NUMBER(8,2)
 - STOCK_QTY – NUMBER
2. Apply the following **table-level constraints**:
 - PRODUCT_ID as PRIMARY KEY
 - PRODUCT_NAME as UNIQUE
 - PRICE should be greater than 0
 - STOCK_QTY should be greater than or equal to 0
3. Insert valid product records into the PRODUCT table.
4. Try inserting a duplicate PRODUCT_NAME and note the error.

Lab Exercise 3: Using ALTER TABLE with Constraints

Aim:

To add and manage constraints on an existing table using the **ALTER TABLE** command.

Question:

Write SQL commands in Oracle to perform the following operations:

1. Create a table **DEPARTMENT** with the following attributes (without constraints):
 - DEPT_ID – NUMBER
 - DEPT_NAME – VARCHAR2(30)
 - LOCATION – VARCHAR2(30)
2. Use the ALTER TABLE command to:
 - Add PRIMARY KEY constraint on DEPT_ID
 - Add UNIQUE constraint on DEPT_NAME
 - Add CHECK constraint to ensure DEPT_ID is greater than 0
3. Insert valid records into the DEPARTMENT table.
4. Attempt to insert a record with duplicate DEPT_NAME and observe the error.

Lab Exercise 4: Enforcing Domain Constraints Using CHECK

Aim:

To enforce **domain integrity** using CHECK constraints in Oracle.

Question:

Write SQL commands in Oracle to perform the following operations:

1. Create a table **EXAM_RESULT** with the following attributes:
 - ROLL_NO – NUMBER (Primary Key)
 - SUBJECT – VARCHAR2(30)
 - MARKS – NUMBER
 - RESULT – VARCHAR2(10)
2. Apply the following constraints:
 - MARKS should be between **0 and 100**
 - RESULT should accept only '**PASS**' or '**FAIL**'
3. Insert valid records into the EXAM_RESULT table.
4. Attempt to insert a record with MARKS = 120 and observe the error.

Lab Exercise 5: Foreign Key Constraint and Referential Integrity

Aim:

To maintain **referential integrity** using FOREIGN KEY constraints.

Question:

Write SQL commands in Oracle to perform the following operations:

1. Create a table **COURSE** with:

- COURSE_ID – NUMBER (Primary Key)
- COURSE_NAME – VARCHAR2(40)
- 2. Create a table **ENROLLMENT** with:
 - ENROLL_ID – NUMBER (Primary Key)
 - STUDENT_NAME – VARCHAR2(50)
 - COURSE_ID – NUMBER (Foreign Key referencing COURSE)
- 3. Insert valid records into both tables.
- 4. Attempt to insert an ENROLLMENT record with a COURSE_ID that does not exist and observe the error.

Lab Exercise 6: NOT NULL and DEFAULT Constraints

Aim:

To apply NOT NULL and DEFAULT constraints to ensure mandatory data entry.

Question:

Write SQL commands in Oracle to perform the following operations:

1. Create a table **USER_ACCOUNT** with the following attributes:
 - USER_ID – NUMBER (Primary Key)
 - USERNAME – VARCHAR2(30) (NOT NULL)
 - PASSWORD – VARCHAR2(30) (NOT NULL)
 - ROLE – VARCHAR2(20) (DEFAULT value should be 'USER')
 - STATUS – VARCHAR2(10) (DEFAULT value should be 'ACTIVE')
2. Insert records without specifying ROLE and STATUS.
3. Display the table and observe default values.
4. Attempt to insert a record with NULL USERNAME and observe the error.

Lab Exercise 7: Composite (Multiple Column) Constraints

Aim:

To implement **composite UNIQUE and PRIMARY KEY constraints**.

Question:

Write SQL commands in Oracle to perform the following operations:

1. Create a table **ATTENDANCE** with:
 - STUDENT_ID – NUMBER
 - SUBJECT_CODE – VARCHAR2(10)
 - ATTEND_DATE – DATE
 - STATUS – VARCHAR2(10)
2. Apply a **composite PRIMARY KEY** on (STUDENT_ID, SUBJECT_CODE, ATTEND_DATE).

3. Insert valid attendance records.
4. Attempt to insert a duplicate record and observe the constraint violation.

Lab Exercise 8: Dropping and Disabling Constraints

Aim:

To manage constraints using DROP and DISABLE options.

Question:

Write SQL commands in Oracle to perform the following operations:

1. Create a table **LIBRARY_BOOK** with:
 - BOOK_ID – NUMBER (Primary Key)
 - BOOK_NAME – VARCHAR2(50) (UNIQUE)
 - PRICE – NUMBER (CHECK: PRICE > 0)
2. Disable the CHECK constraint on PRICE.
3. Insert a record with negative PRICE.
4. Enable the CHECK constraint again and observe the behavior.

Lab Exercise 9: ALTER TABLE with Multiple Constraints

Aim:

To add multiple constraints to an existing table using ALTER TABLE.

Question:

Write SQL commands in Oracle to perform the following operations:

1. Create a table **SUPPLIER** without constraints:
 - SUPPLIER_ID – NUMBER
 - SUPPLIER_NAME – VARCHAR2(40)
 - CONTACT_NO – VARCHAR2(15)
2. Use ALTER TABLE to:
 - Add PRIMARY KEY on SUPPLIER_ID
 - Add UNIQUE constraint on CONTACT_NO
 - Add NOT NULL constraint on SUPPLIER_NAME
3. Insert valid and invalid records and observe constraint violations.